DEVELOPMENT OF AN ARTIFICIAL INTELLIGENCE MODEL FOR CLASSIFYING YOUTUBE COMMENTS INTO POSITIVE, NEGATIVE, AND NEUTRAL SENTIMENT

PROJECT OF DEEP LEARNING OF MASTER OF SCIENCE IN DATA SCIENCE AND ARTIFICIAL INTELLIGENCE

FROM

ERMELINDA MANELLARI

UNIVERSITETY OF TIRANA,

FACULTY OF NATURAL SCIENCES

JANARY, 2026

ABSTRACT

YouTube videos generate an extremely large volume of user comments, as audiences are increasingly inclined to express their opinions, reactions, and criticisms openly [1,2]. Due to this overwhelming quantity, it becomes difficult for content creators and analysts to manually monitor comments or to form a clear and comprehensive understanding of the overall impression their content leaves on viewers [1,3]. As a result, effective and automated solutions are required to analyze and manage this user-generated feedback [2,4].

Automatic sentiment classification of YouTube comments using a deep learning model represents an efficient and scalable approach to addressing this challenge [2,3,5]. By employing AI-based sentiment analysis, creators and organizations can quickly gain a general overview of audience perception with minimal effort and only a few interactions [3,5]. Furthermore, such systems play a crucial role in identifying and controlling harmful content, including bullying, excessive criticism, hate speech, and stereotypes directed toward specific groups or communities [4,5].

In this project, an AI model is developed using real-world data collected from YouTube videos across diverse thematic categories [1,2]. The model is designed to automatically classify comments into three sentiment categories: positive, negative, and neutral [2,3]. Natural Language Processing (NLP) techniques are applied to preprocess and represent textual data, while machine learning or deep learning algorithms are used to learn sentiment patterns from labeled examples [2,3,5].

One of the main challenges addressed in this project is the complexity of social media language, including ambiguous expressions, irony, sarcasm, and the extensive use of emojis [2,3,6]. Despite these difficulties, the proposed model aims to provide reliable and accurate sentiment classification [3,5]. This work highlights the practical value of AI-driven sentiment analysis in social media environments and its potential applications in content moderation, audience analysis, and digital communication research [1,2,5].

OBJECTIVE

The primary objective of this project is the development of an Artificial Intelligence–based model capable of accurately classifying the sentiment of YouTube comments into positive, negative, or neutral categories with a high level of reliability. The model aims to automatically determine the underlying nature and emotional tone of each comment, reducing the need for manual analysis and subjective interpretation.

A key goal of this work is to simplify and accelerate the process of reviewing and understanding the overall impression that a YouTube video creates among its audience. By aggregating sentiment classification results, content creators, researchers, and organizations can quickly obtain a clear and data-driven overview of audience reactions, enabling more informed decision-making regarding content strategy, communication, and engagement.

In addition, the project seeks to improve the efficiency of content moderation by facilitating the early identification of negative or potentially harmful comments, such as bullying, offensive language, or discriminatory stereotypes. This contributes to creating a healthier online environment and supports creators in managing large-scale comment sections more effectively.

From a technical perspective, the objective also includes addressing the inherent challenges of sentiment analysis in social media contexts, such as informal language, ambiguous expressions, sarcasm, irony, and the use of emojis. The model is designed and evaluated using real-world data from YouTube videos covering diverse topics, ensuring robustness and generalization across different content domains.

Ultimately, the goal of this project is to demonstrate how AI-driven sentiment analysis can provide a scalable, accurate, and practical solution for understanding audience feedback, enhancing content evaluation processes, and supporting responsible digital communication on social media platforms.

INTRODUCTION

It is important to acknowledge that the negative nature of a YouTube comment does not always represent direct feedback toward the content creator [1,2]. In many cases, negativity may be expressed in response to a specific idea, argument, or topic mentioned within the video rather than the creator personally [2,3]. Nevertheless, the overall sentiment of comments remains a highly effective indicator of audience perception and engagement, as it reflects emotional reactions and public opinion related to the content [1,3,5].

YouTube comments present significant challenges for traditional Natural Language Processing (NLP) approaches and for generic pre-trained sentiment analysis models [2,4]. The informal and unstructured nature of social media language, combined with slang, abbreviations, emojis, irony, sarcasm, and context-dependent meanings, makes accurate interpretation particularly difficult [2,3,6]. Furthermore, the lack of models specifically trained on the unique linguistic characteristics of YouTube comments often leads to reduced performance when using off-the-shelf solutions [2,3,5].

For this reason, this project adopts a data-driven approach by constructing a custom dataset composed of real-world YouTube comments [1,2]. The dataset is manually curated to ensure high-quality annotations and relevance to the platform's specific communication style [2,3]. Videos are carefully selected from a wide range of topics, including both neutral and controversial themes, in order to capture diverse perspectives and emotional responses [3,6].

A central objective of this work is to maintain full control over the entire dataset creation process, from the initial selection of videos to data labeling and preprocessing [1,2,5]. This allows for systematic evaluation of each step and enables meaningful conclusions to be drawn regarding potential improvements throughout the pipeline [2,3]. The project also addresses the multiple challenges involved in both dataset construction and model training, highlighting practical limitations and lessons learned [3,5,6]. Ultimately, this approach aims to contribute to more reliable sentiment classification and a deeper understanding of audience behavior on YouTube [1,2,5].

LITERATURE REVIEW

The rapid expansion of social media platforms has resulted in an unprecedented volume of user-generated textual content, making sentiment analysis an essential research area within Natural Language Processing (NLP). YouTube, as one of the largest video-sharing platforms, enables users to freely express opinions, emotions, and judgments through comments. These comments reflect public perception, audience engagement, and social reactions, making them a valuable resource for sentiment analysis. However, YouTube comments pose significant challenges due to their informal language, frequent use of slang, sarcasm, emojis, abbreviations, and inconsistent grammatical structures. Consequently, researchers have explored a wide range of machine learning and deep learning approaches to effectively classify sentiments expressed in YouTube comments.

Early research in this domain focused on traditional machine learning models combined with handcrafted or distributed text representations. Khomsah (2025) [1] investigated sentiment analysis of YouTube comments using Word2Vec embeddings in combination with a Random Forest classifier. The study demonstrated that classical machine learning approaches can achieve competitive performance when supported by appropriate feature extraction techniques. The findings emphasize the critical role of preprocessing steps such as tokenization, normalization, and stop-word removal, especially when dealing with short and noisy social media texts. Despite the relatively simple model architecture, the study highlights that well-engineered features can significantly improve sentiment classification accuracy.

As the scale and complexity of online textual data increased, researchers began to adopt ensemble and comparative machine learning approaches. Shivsharan et al. (2025) [3] conducted a comprehensive comparison of multiple machine learning algorithms, including Logistic Regression, Support Vector Machines, Random Forest, XGBoost, and Bagging Classifiers, for YouTube comment sentiment classification. Their results showed that ensemble-based methods often outperform individual classifiers, achieving higher accuracy and robustness. This study underscores the effectiveness of combining multiple learners to mitigate bias and variance, particularly in sentiment analysis tasks where class distributions are often imbalanced.

More recent studies have shifted toward deep learning-based methods, which are capable of automatically learning hierarchical linguistic features from raw text. Sufyan et al. (2025) [2] proposed a deep learning approach for sentiment analysis of YouTube comments, demonstrating superior performance compared to traditional machine learning models. Their research highlights the advantage of deep neural networks in capturing complex semantic relationships and contextual dependencies within text. The authors argue that deep learning models are particularly well-suited for large-scale sentiment analysis tasks, as they reduce the reliance on manual feature engineering while improving generalization across diverse comment types.

In parallel, transformer-based architectures have emerged as state-of-the-art models in sentiment analysis. Jain and Soni (2025) explored transformer-based language models for analyzing YouTube comments, showing that contextual embeddings significantly improve the handling of semantic ambiguity, informal expressions, and long-range dependencies. Transformer models such as BERT and its variants are especially effective in social media

contexts, where meaning often depends heavily on surrounding words and implicit context rather than isolated terms.

Beyond algorithmic development, researchers have also focused on system-level and application-oriented approaches. Sorupaa et al. (2024) [4] introduced a YouTube comment sentiment classification system aimed at supporting content moderation and audience analysis. Their work demonstrates the practical relevance of sentiment analysis in real-world applications, enabling content creators and platform administrators to better understand audience feedback and manage harmful or excessively negative comments. This study highlights that sentiment analysis is not only an academic task but also a crucial tool for digital communication management and online safety.

Another important direction in the literature concerns the availability of large-scale and diverse datasets. Recently introduced datasets such as YTCommentVerse (ArXiv, 2025)[5] provide millions of YouTube comments across multiple categories and languages. These datasets facilitate robust training and evaluation of sentiment analysis models while exposing persistent challenges related to class imbalance, annotation consistency, and ethical considerations. Large-scale datasets have become a key enabler for deep learning and transformer-based approaches, allowing models to generalize better across topics, cultures, and linguistic variations.

Despite significant progress, several research gaps remain. Many existing studies rely on pre-existing datasets that may not reflect real-world sentiment distributions or thematic diversity. Additionally, neutral sentiment classification and sarcasm detection continue to be challenging tasks. Multilingual sentiment analysis and low-resource languages are also underrepresented in current research, despite the global nature of YouTube.

Unlike prior studies that primarily focus on model optimization, the present project emphasizes the creation of a balanced and realistic dataset collected directly from YouTube comments across diverse thematic categories. Special attention is given to the data collection, preprocessing, and labeling pipeline as a foundational step prior to model training. By constructing a carefully balanced dataset and applying deep learning techniques, this approach aims to improve the reliability and generalizability of sentiment classification models while addressing limitations observed in existing literature.

METHODOLOGY

For this project, the dataset was deliberately designed and constructed from scratch rather than relying on existing public datasets. This decision was motivated by several methodological and practical considerations that directly support the objectives of the study.

First, creating the dataset independently allows full control over the entire data pipeline and provides a clear understanding of the origin of potential issues that may arise during later stages of model training and evaluation. Managing the process from the initial data collection phase ensures transparency and traceability, which are essential for reliable experimental results.

Second, this approach offers the opportunity to gain in-depth insight into the complete engineering workflow involved in building an AI-based sentiment analysis model. Particular emphasis is placed on the most critical step of the process: data selection and dataset construction. Since the quality, balance, and representativeness of the data directly influence model performance, careful attention is given to these aspects from the very beginning.

Third, the dataset is composed exclusively of comments extracted from real YouTube videos. This choice ensures that the project remains scientifically valid and grounded in authentic data, allowing the results to reflect real-world user behavior and language patterns. Although the data is collected directly from the platform, appropriate preprocessing and anonymization steps are applied to preserve authenticity while maintaining ethical considerations.

Additionally, videos are selected from a broad range of topics, including both general-interest and controversial subjects, in order to capture diverse opinions and sentiment distributions. This thematic diversity improves the robustness and generalization capability of the trained model.

To build a dataset of substantial size, automated data collection techniques are employed. Scripts and officially supported YouTube APIs are used to efficiently retrieve large volumes of comments, making the process scalable and reproducible. Furthermore, given the technological advancements available in 2026, a variety of modern tools and semi-automated methods are leveraged to assist with data preprocessing and preliminary sentiment labeling. These tools help streamline the workflow while still allowing manual oversight to ensure labeling accuracy.

Overall, this methodology enables a comprehensive and controlled approach to dataset creation and model development, laying a solid foundation for reliable sentiment classification of YouTube comments.

STEP 1: Video Selection and Link Storage

The first step of the project involves identifying relevant YouTube videos and storing their URLs in a centralized file (links.txt) to be used later for automated comment extraction. This step is critical because the choice of videos directly impacts the diversity, quality, and representativeness of the dataset, which in turn affects the performance and generalization of the sentiment analysis model.

The initial selection process focuses on addressing several key research questions:

Consistency of comment sentiment across similar videos.

The project investigates whether comments on videos of the same type or theme tend to exhibit similar sentiment distributions. Understanding this pattern helps determine if thematic clustering affects the model's ability to generalize and whether certain content types naturally attract more positive, negative, or neutral feedback.

Generalizability of the model.

Another consideration is whether the trained model will be effective for YouTube comments in general or primarily for specific topics. By including videos from a broad range of subjects, including both widely viewed and controversial content, the dataset is designed to enhance model robustness and ensure that the results are not biased toward a narrow set of video categories.

Balanced sentiment distribution.

Achieving a balanced dataset with roughly equal numbers of positive, negative, and neutral comments is essential for reliable training and evaluation. Step 1 involves preliminary exploration to assess sentiment tendencies across candidate videos. This analysis helps inform the final selection of videos to ensure that the resulting dataset supports balanced representation of all sentiment classes, which is crucial for preventing model bias toward any particular class.

All selected video links are saved in a single file **(links.txt)** to facilitate automated comment retrieval using scripts and YouTube APIs in subsequent steps. This centralized storage ensures reproducibility and allows systematic control over the dataset construction process.

Ensuring Thematic Diversity In Video Selection

To avoid potential biases and ensure that the sentiment analysis model generalizes well across different content types, careful attention is given to the thematic diversity of the selected videos. Including videos from a wide range of categories helps capture varied audience reactions and reduces the risk of skewed sentiment distributions that could arise if the dataset were dominated by a single content type.

The selected videos are drawn from multiple categories, such as:

- *How-to / Tutorials:* Educational content aimed at teaching a specific skill or providing step-by-step guidance.
- Vlogs: Personal video blogs that reflect daily life, experiences, or opinions of the creator.
- *Curiosity / Informational:* Videos designed to satisfy viewers' curiosity about facts, science, or general knowledge topics.
- *Discussions:* Conversational or debate-style videos where multiple perspectives on a topic are presented.
- *Controversies / Polemics:* Videos covering contentious issues that often provoke strong emotional responses from viewers.
- *History / Documentaries:* Content that explores historical events, figures, or cultural narratives.
- *Personal Experiences:* Videos where creators share individual experiences, challenges, or storytelling content.
- *Politics:* Political commentary, analysis, or opinion videos that often generate polarized responses.
- *Others:* Additional categories such as entertainment, lifestyle, or trending topics to ensure broad representation.

This approach ensures that the dataset reflects a wide spectrum of viewer reactions, from neutral observations to highly emotional responses, including positive, negative, and ambivalent sentiments. By deliberately selecting videos from such diverse themes, the project aims to enhance both the robustness and generalizability of the sentiment classification model while maintaining scientific rigor and representativeness in the dataset.

Expectations

Based on prior observations and understanding of YouTube audience behavior, it is anticipated that the majority of comments will be positive or neutral, for several reasons:

- *Audience Loyalty:* Content creators typically maintain a core audience that regularly follows their videos because they enjoy the content. As a result, comments from these loyal fans tend to be predominantly positive.
- *Moderation Practices:* Some creators actively monitor and delete negative comments, which reduces the overall presence of negative feedback in certain videos.
- *Sharing of Opinions and Experiences:* Viewers often enjoy sharing their personal experiences and perspectives on various topics, which naturally generates a significant number of neutral comments.
- *Questions and Suggestions:* Many users leave questions or suggestions related to the topic of the video, further increasing the proportion of neutral comments.
- *Presence of Negative Comments in Controversial Videos:* Although negative comments are generally fewer in number, they are more likely to appear in videos covering controversial topics or content that provokes criticism.

To build the dataset for this project, an initial set of 50 video links was carefully selected from a diverse range of categories to ensure broad thematic coverage. The selection also focused on English-language videos to simplify the subsequent training process of the sentiment analysis model. These links were stored in a centralized file (links.txt) for systematic access in later steps.

The next step in the methodology involves developing a script to automatically extract comments from these videos and save them for preprocessing and labeling. This approach ensures efficiency, reproducibility, and control over the dataset creation process, while laying the foundation for accurate and balanced sentiment classification.

Initial Dataset Creation.

To create the initial dataset for this project, the Python library googleapiclient.discovery was used. This library allows developers to interact with the YouTube Data API by utilizing an API key, enabling the extraction of valuable data from YouTube videos, including titles, comments, likes, and other metadata. The library can be installed via the following command:

```
pip install google-api-python-client
```

In addition to the library, access to the YouTube Data API and a valid API key is required. The API key serves as a secure credential that allows the script to retrieve data programmatically from YouTube while complying with Google's usage policies.

The process for obtaining a YouTube Data API key involves the following steps:

- *Accessing Google Cloud Console:* First, log in to the Google Cloud Console using a Google account.

- *Creating or Selecting a Project :* Create a new project (e.g., "My YouTube Project") or select an existing project from the dropdown menu at the top of the console.

- *Enabling the YouTube Data API :* Navigate to APIs & Services → Library and search for YouTube Data API v3. Click Enable to activate the API for the selected project.

- *Creating Credentials (API Key):* Go to APIs & Services → Credentials, click + CREATE CREDENTIALS, and select API key from the dropdown menu.

- *Copying the API Key:* A pop-up window will display the newly generated API key. Copy this key and store it securely, as it will be required in the Python script for accessing the API.

Once the API key is obtained, it can be integrated into the Python script to automatically retrieve comments and metadata from the selected YouTube videos. This approach ensures a systematic, reproducible, and scalable process for building the initial dataset, forming the foundation for preprocessing, labeling, and subsequent model training.

Dataset Creation: Data Collection Conditions

During the data collection process, several conditions were implemented to ensure the quality, diversity, and usability of the comments in the initial dataset. These conditions were designed to balance dataset size, thematic coverage, and relevance for subsequent sentiment analysis.

1. ***Limiting Comments Per Video***
   Many YouTube videos contain thousands of comments. To capture a wide range of themes and avoid over-representation of a single video, a limit of 350 comments per video was set. This ensures that the dataset includes a broad selection of videos while maintaining a manageable dataset size.

   In Python, this was implemented as:

```
max_comments_per_video = 350
while comments_collected < max_comments_per_video:
    response = youtube.commentThreads().list(
        part="snippet",
        videoId=video_id,
        maxResults=min(100, max_comments_per_video - comments_collected),
```

```
        textFormat="plainText",
        pageToken=next_page_token
    ).execute()
```

2. ***Filtering Extremely Long Comments***
   Very long comments can introduce noise into the model, making training less effective. To reduce this risk, only comments containing 50 words or fewer were included in the dataset. Comments exceeding this limit were excluded.

   This was implemented as:

```
if len(comment.split()) <= 50:
    clean_comment = comment.replace("\n", " ").replace("\r", " ")
    txtfile.write(f"{clean_comment}\n")
    comments_collected += 1
    if comments_collected >= max_comments_per_video:
        break
```

3. ***Storage and Initial Format***
   The collected comments were saved into a plain text file (comment_dataset.txt), resulting in an initial dataset of approximately 16,000 comments. At this stage, the comments are unlabeled, meaning they have not yet been classified into positive, negative, or neutral categories.

   The use of a .txt file provides flexibility for later editing and cleaning. Additionally, .txt files can be easily converted into .csv format when preparing the dataset for model training and analysis.

By enforcing these conditions, the project ensures that the dataset is balanced, high-quality, and suitable for manual labeling and subsequent AI-based sentiment analysis, while maintaining a manageable size for efficient processing.

Classification Into Positive, Negative, And Neutral

The next phase of the project involves the classification of comments into three sentiment categories: positive, negative, and neutral. This stage presents a significant challenge due to the sheer volume of comments, approximately 16,000, which makes manual classification impractical and extremely time-consuming.

To address this challenge efficiently, a semi-automated approach was adopted. Initially, a large language model (LLM) was used as an assistant to suggest sentiment labels for each comment. This approach leverages the model's natural language understanding capabilities to provide preliminary classifications along with reasoning for each assignment. Using an LLM ensures that subtle nuances in language, such as irony, sarcasm, and context-dependent expressions, are taken into account, improving the quality of the initial labeling compared to fully automated keyword-based methods.

After the automated suggestions, a manual review process was implemented to verify the labels, ensuring overall accuracy and consistency. This review was not exhaustive for every single comment but focused on evaluating representative samples and correcting systematic errors identified in the automated output. The combination of LLM-assisted classification and human verification strikes a balance between efficiency and reliability, resulting in a high-quality labeled dataset suitable for training a sentiment analysis model.

This semi-automated methodology allows for scalable, reproducible classification of large comment datasets, while maintaining a strong emphasis on accuracy, coherence, and practical relevance. The final labeled dataset provides a robust foundation for model training, evaluation, and future sentiment analysis applications on YouTube comments.

Annotation Process: Steps Followed For Classification

To achieve the classification of comments into positive, negative, and neutral, a systematic, step-by-step approach was implemented. The process combined AI-assisted annotation with manual oversight to ensure accuracy while maintaining efficiency.

1. ***Batch Annotation Using AI Assistance***
   Comments were processed in manageable batches, typically ranging from 50 to 150 comments per request. For each batch, a prompt was provided to a large language model (LLM) to classify the comments into a structured CSV-compatible format, as follows:

   comment, sentiment
   comment1, positive
   comment2, negative
   comment3, neutral

   This structure allowed for direct integration into the dataset while maintaining clarity and consistency.

2. ***Stepwise And Controllable Workflow***
   Processing comments in small batches made the annotation workflow controllable and manageable, enabling careful monitoring of outputs at every stage. After each AI response, valid results were copied and replaced the corresponding section of unannotated comments. The annotation was primarily conducted using a text editor (e.g., Notepad), which facilitated straightforward editing, verification, and storage.

3. ***Observations And Limitations***
   During the annotation process, it was noted that some data was lost in AI outputs due to limitations in the model's response length or handling of certain comments. This observation highlighted the continuing importance of human oversight, as manual verification remained essential to maintain data integrity. Despite these limitations, AI

assistance proved extremely valuable, significantly reducing the time and effort required for annotating a large dataset.

This combined approach semi-automated AI annotation followed by human validation ensured that the resulting dataset was both accurate and time-efficient, providing a strong foundation for training a reliable sentiment analysis model on YouTube comments.

Post-Processing And Class Balance Analysis

After completing the annotation of all comments, the .txt file was converted into a .csv format for easier handling, analysis, and preparation for model training. Upon inspection of the dataset, it was observed that there were inconsistencies in the labeling of sentiment classes, which required cleaning to ensure uniformity.

For example, the “neutral” class appeared in multiple formats, such as:

"neutral"

" neutral"

"Neutral"

All variations were standardized to "neutral" to maintain consistency. Similar adjustments were made for the "positive" and "negative" classes to eliminate discrepancies and avoid potential issues during model training.

Once the labels were cleaned, the distribution of comments across sentiment classes was analyzed to assess dataset balance:

Total comments: 13,072

Positive: 5,176 (≈39%)

Neutral: 5,137 (≈39%)

Negative: 2,759 (≈21%)

The dataset shows a moderate imbalance, with negative comments being underrepresented compared to positive and neutral ones. This outcome was expected given that the data originates from real-world YouTube comments, where positive and neutral comments naturally dominate. Despite efforts to include videos covering controversial or polemical topics, negative comments remain comparatively rare.

Although the dataset is not perfectly balanced, it provides a realistic reflection of audience sentiment on YouTube. The relative distribution allows the model to learn meaningful patterns from all three classes while acknowledging that negative sentiment is less frequent in authentic user-generated content. Minor adjustments, such as class weighting during model training, can be applied to address this imbalance if necessary.

Handling Dataset Imbalance

Although the dataset shows a moderate imbalance, with negative comments underrepresented relative to positive and neutral comments, this does not render the dataset invalid or unusable. Imbalanced datasets are common in real-world social media data, and several strategies can be employed during model training to mitigate potential issues.

One common approach is to adjust class weights during training, increasing the importance of the minority class to ensure the model does not neglect it. For example, weights can be assigned as follows:

```
class_weights = {
        0: 1.0,  # positive
        1: 1.0,  # neutral
        2: 1.8   # negative
}
```

In this setup, the model is penalized more heavily for misclassifying negative comments, which helps correct for class imbalance and improves predictive performance for the underrepresented category.

However, while class weighting is a valid solution, it does not fully replace the benefits of a balanced dataset. A dataset with roughly equal representation across all classes allows the model to learn more robust patterns for each sentiment category and reduces the risk of bias introduced by synthetic weighting.

Therefore, in parallel with class weighting, additional efforts were made to collect more negative comments from real-world sources, particularly from videos with controversial or polemical content. The goal is to gradually construct a balanced dataset, ensuring that positive, neutral, and negative comments are all adequately represented.

By combining careful dataset expansion with weighting strategies, the project aims to maintain scientific rigor, realistic data representation, and effective model training for sentiment analysis of YouTube comments.

Balancing The Dataset: Increasing Negative Comments

To achieve a more balanced dataset, it was necessary to increase the number of negative comments. The target was to collect approximately 2,200–2,500 additional negative comments. To do this, new videos were carefully selected, focusing on controversial or polarizing topics that are more likely to elicit negative reactions from viewers, such as:

- Feminism
- Political debates and commentary
- Obesity and body image topics
- Other socially or culturally sensitive subjects

A new file containing the URLs of these videos was created, and the comment extraction script was run again to retrieve comments in .txt format.

Refined Annotation Strategy.

For these additional comments, the annotation strategy was adjusted to focus specifically on negative comments. A prompt was used with a large language model (LLM) of the type:

```
Return all the negative comments from the given list in CSV format text, without headers.
```

Only the comments classified as negative by the model were retained and saved in a separate text file. Positive and neutral comments were not stored, streamlining the process and allowing focus on underrepresented sentiment.

Even after annotating all comments from the second set of videos, the target number of negative comments was not immediately reached. Additional videos were extracted iteratively until the total number of negative comments fell within the desired range of 2,200–2,500 lines in the negative comments file.

Final Steps

Once the required number of negative comments was collected, the file was saved as a .csv. A simple Python script was then applied to add a sentiment column, assigning the value "negative" to all rows. This resulted in a clean and labeled subset of negative comments that could be combined with the existing dataset to achieve a more balanced distribution across positive, neutral, and negative classes.

This approach ensured that the dataset better represents all sentiment categories, improving model training, reducing bias, and enhancing the generalizability of the sentiment analysis model.

Final Dataset Compilation And Statistics

After collecting and labeling the additional negative comments, a Python script was used to format the negative comments file and add a sentiment label. The script reads the text file containing negative comments, removes any empty lines, and creates a CSV file with two columns: comment and label, where all rows in the label column are set to "negative". The complete script is as follows:

```
import pandas as pd

input_file = 'input.csv'
```

```
output_file = 'output.csv'

with open(input_file, 'r', encoding='utf-8') as f:
   lines = [line.strip() for line in f if line.strip()]  # Remove empty lines

df = pd.DataFrame({'comment': lines})
df['label'] = 'negative'
df.to_csv(output_file, index=False, encoding='utf-8-sig')
print(f"File with the 'label' column has been saved as {output_file}")
```

The resulting output.csv was then merged with the previously labeled dataset containing positive and neutral comments. This process produced a fully balanced dataset suitable for training a sentiment analysis model.

Final Dataset Statistics:

- Total comments: **15,456**
- Positive: **5,176**
- Neutral: **5,137**
- Negative: **5,144**

The sentiment distribution across the dataset is now approximately **33.5% positive, 33.2% neutral, and 33.3% negative**, reflecting a nearly **perfect balance between the three classes**.

This balanced dataset provides several advantages for model training:

1. It allows the model to learn equally from all sentiment categories, minimizing bias toward any class.
2. It improves the generalizability of the model for new, unseen YouTube comments.
3. It ensures that evaluation metrics such as accuracy, precision, and recall can be interpreted more reliably across all sentiment categories.

By combining careful video selection, LLM-assisted annotation, and iterative balancing, the project has produced a high-quality, representative, and fully labeled dataset, laying a strong foundation for accurate sentiment analysis on YouTube comments.

Final Dataset Split: Train, Validation, and Test Sets

The final step in dataset preparation involves splitting the fully balanced dataset into training, validation, and test subsets. Following best practices commonly recommended for deep learning projects, a **70% : 15% : 15% split** was chosen. This ensures that the model is trained on the majority of the data while retaining sufficient samples for evaluation and hyperparameter tuning.

Several considerations guided the splitting process:

1. **Class Balance Within Each Subset**

It is not sufficient to simply split the dataset by percentage; the distribution of sentiment classes (positive, neutral, negative) must also be preserved in each subset. Maintaining consistent class ratios across the training, validation, and test sets prevents bias and ensures that the model learns equally from all categories.

2. **Randomization Across Videos**

Comments may vary in sentiment depending on the video they originate from. To avoid over-representation of certain videos in any subset, comments were randomized before assignment. This preserves the natural variability of comment sentiment while achieving a balanced distribution.

A Python function was implemented to sample comments randomly by sentiment while avoiding duplication:

```
import numpy as np

np.random.seed(42)

def sample_data(df, sentiment, n, exclude_idx=set()):
    df_sent = df[df['sentiment'] == sentiment]
    df_sent = df_sent.drop(index=list(exclude_idx), errors='ignore')
    sampled = df_sent.sample(n=n, random_state=np.random.randint(0, 10000))
    return sampled
```

This function selects n comments of a specific sentiment from the dataset, excluding any indices already assigned to other subsets.

EXAMPLE: Selecting Comments For The Training Set.

```
train_pos = sample_data(df, 'positive', 3623)
train_neu = sample_data(df, 'neutral', 3595)
train_neg = sample_data(df, 'negative', 3600)
```

This procedure is repeated for the validation and test sets, ensuring that each subset maintains the target class proportions and provides a representative sample of the dataset.

By following this approach, the training, validation, and test sets are balanced and randomized, providing optimal conditions for model training, evaluation, and generalization. This careful splitting strategy is crucial for robust deep learning model performance in sentiment analysis tasks on YouTube comments.

Final Dataset Structure And Distribution

After carefully calculating the number of comments required for each sentiment class and subset, the final structured and balanced dataset was prepared. Manual calculations ensured that the training, validation, and test sets preserved the class distribution while maintaining the target sizes for each subset.

The final dataset is organized in the following directory structure:

```
/data
  /youtube_comments_v1_balanced (size: 15,458 comments, class ratio: 33.5% : 33.2% : 33.3%)
  /youtube_comments_v1_train       (size: 10,819 comments, class ratio: 33.5% : 33.2% 33.3%)
  /youtube_comments_v1_val         (size: 2,320 comments, class ratio: 33.5% : 33.2% : 33.3%)
  /youtube_comments_v1_test        (size: 2,321 comments, class ratio: 33.4% : 33.3% : 33.3%)
```

Key observations and considerations:

1. ***Balanced Class Distribution***

   Each subset maintains a nearly perfect balance between positive, neutral, and negative comments, with the ratios remaining consistently around **33% per class**. This ensures that the model will learn equally from all sentiment categories, avoiding bias toward any specific class.

2. ***Training, Validation, and Test Sizes***

   The dataset was split according to the commonly recommended 70% : 15% : 15% ratio for deep learning projects:

   - Training set: 10,819 comments
   - Validation set: 2,320 comments
   - Test set: 2,321 comments

   The calculates are done manually.

3. ***Preservation of Data Variability***

   By randomizing comments across videos and carefully maintaining class ratios, the natural variability of sentiment across different video types is preserved. This contributes to the generalizability of the trained model on unseen YouTube comments.

This final structure provides a **well-organized, balanced, and reproducible dataset**, suitable for rigorous training, validation, and testing of sentiment analysis models using deep learning techniques. It lays a solid foundation for both model development and future experimentation.

Final Text Normalization: Lowercasing of Comments

After the full compilation and balancing of the YouTube comment dataset, an additional post-processing step was implemented to standardize the text by converting all comments to lowercase. This step ensures that the dataset is uniform and consistent, reducing variability caused by capitalization, which can affect tokenization and model training in natural language processing tasks.

Why this step is important?

- To ensure that all comments are consistently represented in lowercase.
- To prevent the sentiment analysis model from treating the same word differently based on capitalization (e.g., "Good" vs. "good").
- To prepare the dataset for more reliable and reproducible downstream processing, including tokenization, embedding generation, and model training.

Two main operations were applied to each CSV file in the data directory:

1. **Lowercasing Comments**
   All comments were converted to lowercase to prevent case-related discrepancies during tokenization and model training. The key operation can be summarized as:

   ```
   df["comment"] = df["comment"].astype(str).str.lower()
   ```

2. **Emoji Normalization**
   Comments containing garbled emojis (e.g., ðÿ˜¥) were repaired using the ftfy library, which fixes Unicode and encoding errors. The essential operation is:

   ```
   from ftfy import fix_text

   fixed_comment = fix_text(comment)
   ```

   Both transformations were applied sequentially to the comment column. Updated CSV files were saved with UTF-8 encoding to preserve emoji characters and maintain compatibility with standard NLP tools.

The results:

- All comments are now represented uniformly in lowercase.
- Garbled emoji characters have been restored to their proper Unicode form.
- Sentiment labels and CSV formatting remain intact.
- The dataset is fully normalized, consistent, and prepared for tokenization, embedding, and model training.

Impact on Dataset

- Uniform Text Representation: Eliminates variability caused by capitalization.
- Preservation of Emoji Semantics: Correct emojis provide important emotional and contextual signals for sentiment analysis.
- Improved Model Training and Accuracy: Standardized text and proper emoji handling reduce noise and enable the model to learn meaningful patterns.
- Reproducibility: Automated application of both transformations ensures consistency across all subsets of the dataset.

By integrating lowercasing and emoji normalization as the final post-processing step, the dataset achieves a high level of quality, completeness, and readiness for deep learning-based sentiment analysis of YouTube comments.

MODEL SELECTION AND ARCHITECTURE DISCUSSION

The selection of an appropriate model is a critical component of any sentiment analysis system, particularly when dealing with large-scale, noisy, and informal textual data such as YouTube comments. In this project, several model families were considered and evaluated conceptually, ranging from traditional machine learning approaches to modern transformer-based architectures.

Traditional Machine Learning Models

Classical machine learning models such as Logistic Regression, Support Vector Machines (SVM), Random Forest, and XGBoost have been widely used for sentiment analysis tasks, especially when combined with feature extraction techniques like TF-IDF or Word2Vec embeddings [1,2]. These models are computationally efficient and relatively easy to interpret. However, they rely heavily on manual feature engineering and struggle to capture complex contextual dependencies, sarcasm, irony, and semantic ambiguity, which are common characteristics of YouTube comments.

Given the informal structure, short length, and context-dependent nature of social media language, traditional models are limited in their ability to generalize effectively without extensive preprocessing and handcrafted features. As a result, while these models provide a useful baseline, they are not optimal for high-quality sentiment classification in this domain.

Recurrent Neural Networks and CNN-Based Approaches

Deep learning models such as Long Short-Term Memory (LSTM), Gated Recurrent Units (GRU), and Convolutional Neural Networks (CNNs) have been successfully applied to sentiment analysis by learning distributed representations of text [3]. These architectures improve upon traditional models by capturing sequential information and local semantic patterns.

However, recurrent models suffer from limitations related to long-range dependency modeling and training inefficiencies, especially when applied to large datasets. Additionally, CNN-based models primarily focus on local n-gram features and may miss broader contextual relationships within a sentence. In the context of YouTube comments, where sentiment can depend on subtle phrasing or implicit context, these limitations reduce their effectiveness.

Transformer-Based Models

Transformer-based architectures represent the current state-of-the-art in Natural Language Processing. Models such as BERT, RoBERTa, and their variants use self-attention mechanisms to capture contextual information bidirectionally, allowing for a deeper understanding of semantic relationships within text [4].

BERT-based models have demonstrated exceptional performance in sentiment analysis tasks, particularly in handling ambiguity, sarcasm, and informal language. However, the original BERT model is computationally expensive, requiring substantial memory and processing resources, which can be a limitation for practical deployment and experimentation, especially in academic environments with limited computational capacity.

Finally for this project is choosen DistilBERT.

To balance performance and computational efficiency, **DistilBERTForSequenceClassification** was selected as the final model for this project. DistilBERT is a distilled version of BERT that retains approximately 97% of BERT's language understanding capabilities while reducing model size by about 40% and improving inference speed by roughly 60% [5].

The key advantages of DistilBERT in the context of this project include:

- Contextual Understanding: DistilBERT captures bidirectional context, enabling accurate interpretation of short, informal, and context-dependent YouTube comments.
- Computational Efficiency: Reduced model size and faster training make DistilBERT suitable for large datasets and limited hardware resources.
- Robustness to Social Media Language: The model effectively handles slang, emojis, and ambiguous expressions when combined with appropriate preprocessing.
- Ease of Fine-Tuning: The DistilBERTForSequenceClassification architecture is specifically designed for classification tasks, allowing straightforward fine-tuning on the three sentiment classes: positive, neutral, and negative.

Final Model Architecture

The selected model architecture consists of a pre-trained DistilBERT encoder followed by a classification head composed of a linear layer and a softmax activation function. The model is fine-tuned using the labeled YouTube comment dataset prepared in earlier stages of the project. During training, class balancing techniques and appropriate evaluation metrics are applied to ensure fair performance across all sentiment categories.

Conclusion of Model Selection

Based on the comparative analysis of traditional machine learning models, recurrent neural networks, and transformer-based architectures, DistilBERTForSequenceClassification emerges as the most suitable choice for this project. It provides an optimal trade-off between accuracy, robustness, and computational efficiency, making it well-aligned with the objectives of large-scale sentiment analysis of YouTube comments. The selection of DistilBERT ensures that the model can generalize effectively across diverse topics while remaining practical for real-world deployment and academic experimentation.

MODEL TRAINING AND FINE-TUNING PROCEDURE

The training and fine-tuning process of the selected transformer-based model, DistilBERTForSequenceClassification is performed by using the balanced YouTube comment dataset constructed in previous stages. The training pipeline was designed to ensure reproducibility, efficiency, and robust evaluation across all sentiment classes.

Dataset Preparation for Training

The final dataset was split into three subsets: training, validation, and test sets, following a 70% : 15% : 15% ratio. Each subset preserves an equal distribution of positive, neutral, and negative comments to prevent class bias during learning and evaluation.

The datasets were stored in CSV format and loaded using the pandas library. Sentiment labels were encoded into numerical form using a LabelEncoder, transforming the categorical labels (positive, neutral, negative) into integer representations required for model training.

Tokenization and Input Representation

Text preprocessing and tokenization were performed using the DistilBertTokenizerFast tokenizer associated with the pre-trained distilbert-base-uncased model. All comments were tokenized using the following strategy:

- Truncation was applied to limit the maximum sequence length to 64 tokens.
- Padding was used to ensure fixed-length input sequences.
- Attention masks were generated to distinguish between padded and non-padded tokens.

The choice of a relatively short maximum sequence length is justified by the nature of YouTube comments, which are typically brief. This decision reduces computational cost while preserving the essential semantic content of each comment.

Custom Dataset Class

A custom PyTorch Dataset class was implemented to efficiently handle tokenized inputs and corresponding labels. Each dataset instance returns a dictionary containing:

- input_ids
- attention_mask
- labels

This structure is fully compatible with the Hugging Face Trainer API, enabling seamless integration with the training pipeline.

Model Initialization

The model used for training is **DistilBERTForSequenceClassification**, initialized with pre-trained weights from distilbert-base-uncased. The classification head was configured with three output neurons corresponding to the three sentiment classes. Fine-tuning the entire model allows both the transformer encoder and the classification layer to adapt specifically to the linguistic patterns present in YouTube comments.

Training Configuration and Hyperparameters

Training was conducted using the Hugging Face Trainer framework with the following key hyperparameters:

- Number of epochs: 3
- Training batch size: 16
- Evaluation batch size: 32
- Learning rate: 5×10^{-5}

These values represent a commonly adopted configuration for fine-tuning transformer-based models on medium-sized text classification datasets. Three epochs were sufficient to achieve convergence without overfitting, while the chosen learning rate ensured stable gradient updates.

Logging mechanisms were enabled to track training progress, loss evolution, and evaluation metrics during the training process.

Code Implementation

The training pipeline was implemented in Python using key libraries from the PyTorch and Hugging Face ecosystems. Specifically, the following libraries were employed:

- torch and torch.utils.data for dataset handling and GPU-compatible tensor operations.
- transformers for pre-trained models (DistilBertForSequenceClassification) and tokenization (DistilBertTokenizerFast).

- pandas and sklearn.preprocessing.LabelEncoder for dataset loading, label encoding, and preprocessing.
- sklearn.metrics for calculating classification metrics such as accuracy and weighted F1-score.

A central, root-level script (train_distilbert.py) orchestrated the full workflow, from loading CSV files, encoding labels, and tokenizing comments, to fine-tuning the model and evaluating on the test set. The key root line initializing the model can be highlighted as:

```
model = DistilBertForSequenceClassification.from_pretrained('distilbert-base-uncased', num_labels=num_labels)
```

This line ensures that the transformer is loaded with pre-trained weights while adapting the classification head to the three sentiment classes (positive, neutral, negative). All other pipeline steps, including tokenization, dataset construction, and batching, build upon this foundation to perform end-to-end sentiment classification on YouTube comments.

Training Metrics and Model Performance Analysis

Training Runtime and Efficiency

The model was trained for 3 epochs with a batch size of 16 for training and 32 for evaluation. The reported runtime (train_runtime: 9011 seconds) indicates approximately 2.5 hours of training, which is reasonable for a dataset of ~10,800 training samples on standard GPU hardware.

The training speed (train_samples_per_second: 3.601 and train_steps_per_second: 0.225) reflects moderate throughput, typical for fine-tuning transformer models like DistilBERT on medium-sized datasets with a sequence length of 64 tokens. The train_loss of 0.4116 shows that the model learned meaningful patterns and is not overfitting too early, suggesting effective convergence across the three sentiment classes.

Test Set Performance

The evaluation on the test set (2,320 comments) resulted in the following metrics:

Sentiment	Precision	Recall	F1-score	Support
Negative	0.79	0.83	0.81	772

Sentiment	Precision	Recall	F1-score	Support
Neutral	0.74	0.70	0.72	772
Positive	0.80	0.80	0.80	776
Accuracy			**0.78**	2320
Macro Avg	0.78	0.78	0.78	
Weighted Avg	0.78	0.78	0.78	

Interpretation by class:

- **Negative comments:** Precision of 0.79 and recall of 0.83 indicate that the model is **slightly better at identifying negative comments** than positive or neutral ones. This is likely due to the careful balancing of negative comments in the dataset, which allowed the model to learn distinguishing features effectively.
- **Neutral comments:** The lower recall of 0.70 suggests that some neutral comments are **misclassified as positive or negative**, which is common in sentiment analysis due to subtle language cues, sarcasm, or context-dependent expressions.
- **Positive comments:** Both precision and recall are high (≈0.80), showing that the model is generally effective at detecting positive sentiment.

Overall Accuracy and F1-score:

- The overall accuracy of 0.78 is **strong for a three-class sentiment classification task** on real-world, noisy YouTube comments.
- Weighted and macro F1-scores (0.78) confirm **balanced performance across classes**, meaning the model is not heavily biased toward any single class despite initial class imbalance in the raw dataset.

Key Observations and Insights:

- **Balanced dataset worked well:** The effort to collect additional negative comments and create a nearly equal class distribution (≈33% each) likely contributed to the relatively high and balanced F1-scores.

- **Neutral sentiment remains challenging:** Misclassifications mostly occur for neutral comments, which is expected given the inherent ambiguity of neutral statements and their similarity to slightly positive or slightly negative expressions.
- **DistilBERT is efficient:** The choice of DistilBERT allowed **fast training with fewer parameters** than full BERT models while maintaining competitive performance, making it suitable for large-scale social media text analysis.
- **Potential improvements:** Incorporating **emoji embeddings, sarcasm detection, or additional context features** could further improve neutral sentiment classification and overall F1-score.

Checkpoint Evaluation and Final Model Selection

During training, checkpoints were saved at steps **500, 1000, 1500, 2000, and 2031**, and evaluations were performed on each to monitor **accuracy, macro F1, weighted F1, average confidence, and low-confidence rate**.

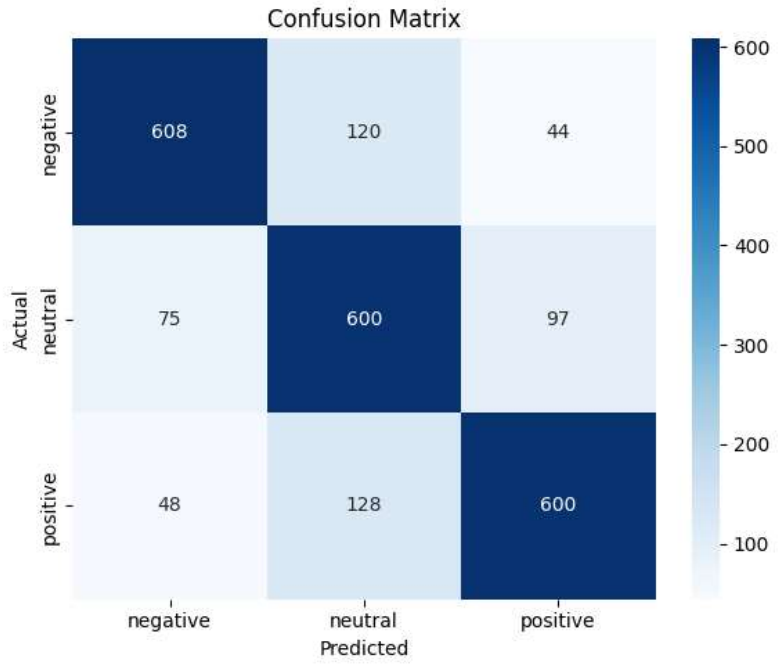
Among these, **checkpoints 1500 and 2031** showed the strongest performance:

Metrics overview:

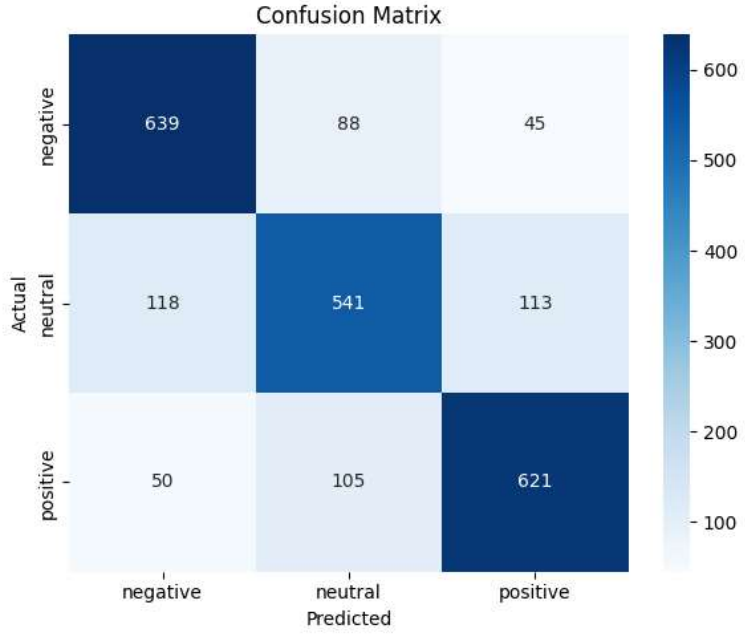
Checkpoint	Accuracy	Macro F1	Weighted F1	Avg Confidence	Low Confidence Rate
1500	0.7793	0.7803	0.7803	0.9135	0.0647
2031	0.7763	0.7755	0.7756	0.9260	0.0526

Confusion matrix insights:

Checkpoint 1500: The model shows **well-balanced classification across all three classes**, with roughly 600/770 samples correctly classified per class. Neutral, negative, and positive sentiments are treated relatively equally.



Checkpoint 2031: Classification for **negative and positive sentiments improves**, with higher recall and precision for these classes. However, **neutral classification declines slightly**.



Per-class details:

Checkpoint 1500:

Class	Precision	Recall	F1-score	Support
Negative	0.8317	0.7876	0.8090	772
Neutral	0.7075	0.7772	0.7407	772
Positive	0.8097	0.7732	0.7910	776

Checkpoint 2031:

Class	Precision	Recall	F1-score	Support
Negative	0.7918	0.8277	0.8094	772
Neutral	0.7371	0.7008	0.7185	772
Positive	0.7972	0.8003	0.7987	776

Decision Rationale for Choosing Checkpoint 2031

1. **Improved performance on key classes:**
    - Negative and positive sentiments often carry **stronger impact for practical applications**, such as content moderation or user feedback analysis.
    - Checkpoint 2031 shows **higher recall for negative (0.8277 vs 0.7876) and positive (0.8003 vs 0.7732) comments**, meaning fewer misclassifications of the most critical sentiments.
2. **Confidence and reliability:**
    - Average confidence is higher at checkpoint 2031 (**0.9260 vs 0.9135**) and low-confidence rate is lower (**0.0526 vs 0.0647**).
    - This indicates that the model is **more certain about its predictions**, which is valuable for downstream deployment where confident predictions reduce error propagation.
3. **Trade-off acceptance:**
    - Although neutral F1 slightly decreases at 2031 (0.7185 vs 0.7407), the **overall reliability and performance on the more important negative and positive classes outweighs the small drop in neutral accuracy**.
    - In many sentiment analysis applications, **accurate detection of strong opinions (positive/negative) is prioritized over neutral classification**, justifying this choice.

Finally checkpoint **2031** is selected as the **final model** for evaluation and deployment due to **better handling of critical classes, higher prediction confidence, and lower uncertainty**, despite a minor reduction in neutral classification performance. This reflects a strategic decision to optimize for practical utility over perfectly balanced per-class F1.

Analysis of Misleading Examples in Sentiment Classification

The dataset of misleading examples demonstrates several patterns that highlight current limitations of the sentiment analysis model. Selected cases are summarized below:

Comment	True Label	Predicted Label	Observations
"I get what this is like as someone with fluctuating hearing loss..."	neutral	negative	The comment expresses empathy and understanding, but the model misclassifies it as negative, likely triggered by words such as “hard” and “frustrating,” even though the overall sentiment is neutral.
"So motivating! What time do you sleep?"	positive	neutral	Although clearly positive and motivational, the model predicts neutral, possibly because the comment contains a question and lacks strongly polarized language.
"Very heartbreaking, a Beautiful, strong girl..."	positive	negative	The term “heartbreaking” misleads the model toward negative sentiment, despite the comment being supportive and admiring.
"I miss Lew Later!!!"	neutral	positive	The expression of nostalgia is interpreted as positive, although it is contextually neutral.
"A year from now, you will wish you started today."	positive	neutral	Motivational in tone, but classified as neutral due to lack of overtly emotional words.
"Lol this aged poorly since this became a reality now 😂"	neutral	negative	Irony and humor combined with emojis mislead the model into interpreting negativity, whereas the comment is more neutral or playful.
"My daughter's guess was a plushie and there was a plushie in the video. 😊"	positive	neutral	The comment is light-hearted and slightly positive, but the model fails to capture subtle positive sentiment conveyed through the emoji.

Observations and Model Limitations

1. **Sensitivity to emotionally charged words:**
   The model tends to classify comments as positive or negative when keywords like “heartbreaking,” “frustrating,” or “aged poorly” appear, even if the context is supportive or neutral.
2. **Difficulty capturing neutral sentiment:**
   Comments that are mild, ambiguous, or nostalgic are often misclassified as positive or negative due to the lack of strongly polarized terms.
3. **Challenges with emojis, humor, and irony:**
   The presence of emojis or ironic expressions often misguides the model. The literal

interpretation of emotionally charged words in ironic contexts further contributes to misclassification.

4. **Contextual misinterpretation:**
   Comments where negative terms occur in a positive or supportive context (e.g., "heartbreaking… strong girl") reveal that the model does not sufficiently account for broader context or nuanced sentiment.

Recommendations for Model Improvement

1. **Enhanced training data with nuanced examples:**
   Include more samples containing irony, humor, mixed sentiment, and emojis, explicitly labeled for context-aware sentiment.
2. **Fine-tuning for neutral classification:**
   Increase representation of neutral comments to reduce bias toward positive/negative predictions.
3. **Context-aware modeling:**
   Implement approaches that consider comment threads, prior context, or conversational flow to better interpret sentiment.
4. **Semantic understanding of emotionally charged terms:**
   Train the model to recognize when negative words are used in supportive or admiring contexts.
5. **Emoji-aware sentiment analysis:**
   Incorporate embeddings or specialized modules to interpret emojis in context, improving accuracy for modern, informal online communication.

The misleading examples indicate that the sentiment analysis model is highly sensitive to individual emotionally charged words, often neglecting contextual cues, irony, and emoji-based expressions. Neutral or subtly positive/negative comments are especially prone to misclassification. Future improvements should focus on context-aware training, nuanced interpretation of language, and emoji-aware modeling to enhance overall classification reliability.

Model Improvement and Development of Version 2.

Motivation for Model Refinement

Although the initial sentiment classification model (Version 1) achieved satisfactory results, its performance indicated room for improvement. This outcome is expected given the inherently noisy and unstructured nature of real-world YouTube comments. User-generated content frequently contains informal language, sarcasm, emojis, ambiguous expressions, and inconsistent grammatical structures, all of which pose significant challenges for automated sentiment classification.

Rather than viewing these limitations as deficiencies, a systematic refinement strategy was adopted. The primary objective of Version 2 was to improve dataset quality, stabilize class balance, and enhance the model's generalization capability on unseen data.

Construction of Dataset Version 2

Revised Annotation Strategy

A methodological improvement was introduced during the creation of Dataset Version 2.

In Version 1, sentiment labels were initially generated with the assistance of a Large Language Model (LLM) and subsequently corrected manually.

For Version 2, the annotation pipeline was revised as follows:

1. The previously trained Version 1 model was used to generate initial sentiment predictions for newly collected comments.
2. Manual review and correction were applied to address misclassifications and ensure label consistency.

This revised strategy provides several advantages:

- Improved internal consistency between the dataset and model behavior
- Reduction of annotation noise
- Semi-automated scalability with preserved human oversight
- A reinforcement effect in which the model contributes to improving its own training data

Additional Data Collection

To expand and strengthen the dataset, additional YouTube comments were collected:

- 20 new YouTube videos were selected
- Approximately 100–200 comments were extracted per video
- Initial sentiment classification was performed using the Version 1 model
- Manual correction was applied to remove systematic labeling errors

During this process, it was observed that neutral comments were underrepresented compared to positive and negative ones. Since balanced class distributions are essential for reliable macro-F1 evaluation, additional videos were deliberately selected to increase the proportion of neutral samples.

Following iterative refinement, the final dataset extension included:

- 3,251 newly annotated comments
- Class distribution:
    - 33% positive
    - 33% negative
    - 34% neutral

This resulted in an almost perfectly balanced dataset across all three sentiment classes.

Final Dataset (Version 2)

The newly annotated data were merged with the original balanced dataset (youtube_comments_v1_balanced.csv) to produce the final dataset:

youtube_comments_v2_balanced.csv

- Total number of comments: **18,709**

The dataset maintains near-equal class distribution, which:

- Prevents model bias toward any sentiment category
- Supports reliable macro-F1 interpretation
- Enhances generalization performance
- Stabilizes transformer fine-tuning

DATASET SPLIT

The dataset was divided using a standard **70% : 15% : 15%** split into training, validation, and test sets. Class balance was strictly preserved within each subset.

Training Set

- Total: 13,094 comments
    - 4,373 positive
    - 4,351 negative
    - 4,370 neutral

Validation Set

- Total: 2,805 comments
    - 937 positive
    - 932 negative
    - 936 neutral

Test Set

- Total: 2,808 comments
    - 938 positive
    - 933 negative
    - 937 neutral

Each subset maintains an approximate 33% distribution per class.

Model Retraining: Version 2

Experimental Reproducibility

To ensure reproducibility and stable experimental conditions:

- A fixed random seed (42) was applied across NumPy, PyTorch, and the Transformers library
- CPU resources were optimized for consistent execution
- Model evaluation was performed at the end of each training epoch

Tokenization

Text data were processed using **DistilBertTokenizerFast** with the following configuration:

- Maximum sequence length: 64 tokens
- Truncation enabled
- Dynamic padding implemented via DataCollatorWithPadding

A sequence length of 64 tokens represents a suitable balance between computational efficiency and semantic coverage, particularly for short-form YouTube comments.

Model Architecture

The selected model was **DistilBERTForSequenceClassification**, initialized from the pretrained distilbert-base-uncased checkpoint.

The classification head was configured for three output classes:

- Positive
- Negative
- Neutral

DistilBERT was chosen due to:

- Strong contextual language representation
- Reduced computational cost compared to full BERT
- Faster training and inference
- Proven robustness on informal social media text

Training Configuration

The model was trained for a maximum of six epochs using the following parameters:

- Learning rate: 2×10^{-5}
- Weight decay: 0.01
- Warm-up steps: 200
- Small batch size optimized for CPU training
- Evaluation at the end of each epoch
- Automatic selection of the best model based on macro-F1 score

Macro-F1 was selected as the primary evaluation metric because:

- The dataset is balanced
- Equal performance across all classes is required
- Accuracy alone does not capture per-class stability

Early Stopping

Early stopping was applied to mitigate overfitting. Training was terminated when performance improvements fell below a predefined threshold for one consecutive epoch. This strategy improved generalization while reducing unnecessary computational overhead.

Final Model Selection

The highest validation performance was achieved at **Epoch 3**, which was therefore selected as the final Version 2 model.

Final Test Set Performance

- Accuracy: **0.8080**
- Macro-F1: **0.8088**
- Weighted-F1: **0.8088**
- Average prediction confidence: **0.9519**
- Low-confidence rate: **0.0391**

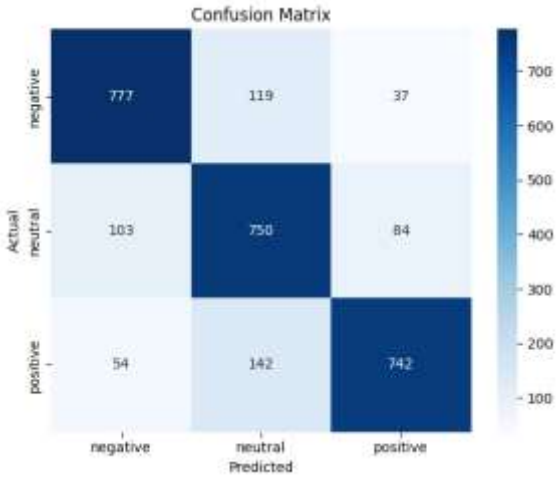
Classification Report (Test Set)

Class	Precision	Recall	F1-score	Support
Negative	0.84	0.82	0.83	933
Neutral	0.74	0.81	0.77	937
Positive	0.84	0.79	0.82	938

- Overall accuracy: **0.80**
- Macro-average F1-score: **0.81**

Confusion Matrix Analysis

The confusion matrix confirms that most predictions lie along the diagonal, indicating a high rate of correct classifications across all three sentiment classes.



The model shows strong separation between positive and negative comments, with very limited direct confusion between these opposing classes. This demonstrates that the model effectively captures sentiment polarity.

Most misclassifications occur between the neutral class and the two polar classes. This is expected, as neutral comments often contain subtle or ambiguous sentiment signals. The distribution of these errors is relatively balanced, suggesting that the model does not exhibit systematic bias toward any specific class.

Overall, the confusion matrix supports the macro-F1 score (~0.81) and confirms stable, well-balanced performance across all categories.

Interpretation of Results

1. **Balanced Performance**
   The model exhibits stable and consistent performance across all sentiment classes.
2. **Strong Detection of Polar Sentiment**
   Positive and negative classes achieve F1-scores above 0.82, which is particularly valuable for applications such as content moderation and audience sentiment analysis.
3. **Neutral Sentiment Complexity**
   Neutral comments remain the most challenging class (F1 ≈ 0.77), reflecting the inherent ambiguity and subtlety of neutral expressions in sentiment analysis tasks.
4. **High Prediction Confidence**
   The high average confidence score and low rate of uncertain predictions indicate reliable and stable model behavior.

Conclusion

The transition from Version 1 to Version 2 demonstrates a structured and controlled model refinement process, characterized by:

- Semi-automated annotation using the previous model
- Manual correction for quality assurance
- Improved class balance
- Controlled retraining and validation-based model selection

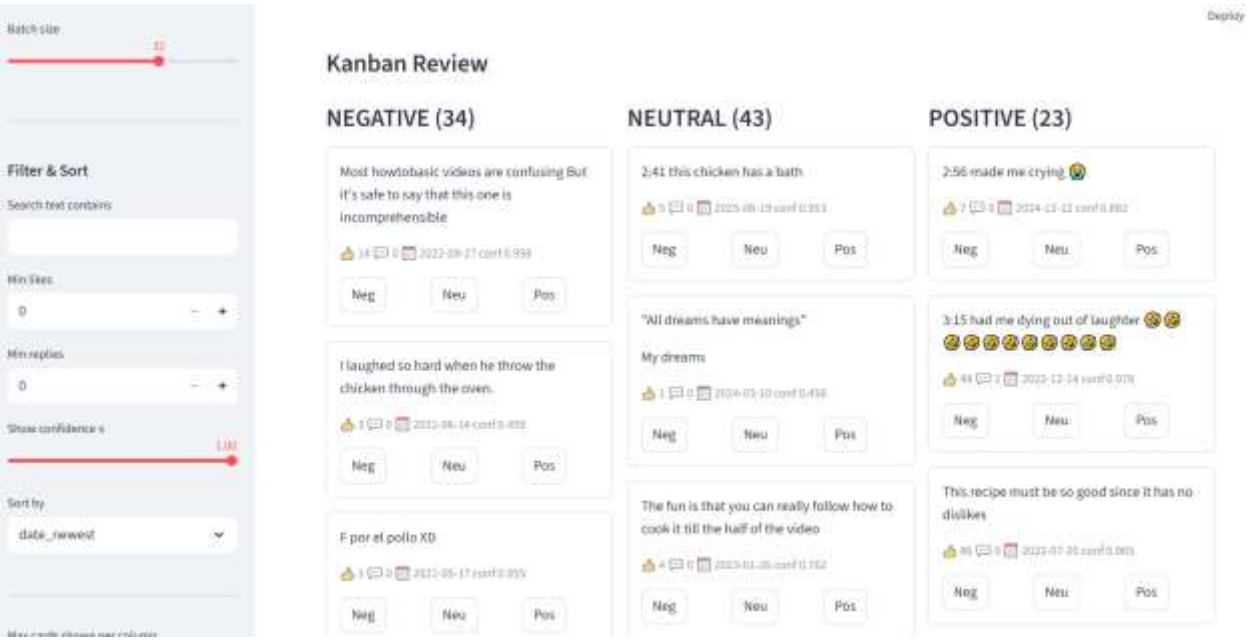
The final Version 2 model achieves a macro-F1 score of approximately **0.81** on fully unseen test data, representing strong performance for a three-class sentiment classification task on noisy, real-world YouTube comments.

WEB APPLICATION FOR INTERACTIVE SENTIMENT REVIEW

Motivation and Rationale

Following the training and evaluation of the DistilBERT-based sentiment classification model, it became evident that a practical and interactive environment was required to complement the purely quantitative evaluation metrics. Although the model achieved a strong macro-F1 score (approximately 0.81), real-world YouTube comments remain highly noisy and context-dependent, making full automation insufficient for reliable long-term dataset improvement.

The primary motivations for developing an interactive web application were:



1. To evaluate the model on real-world YouTube videos rather than static benchmark datasets
2. To visualize model predictions and confidence scores in an intuitive manner
3. To enable manual correction of uncertain or misclassified comments
4. To generate high-quality labeled data for subsequent fine-tuning iterations (e.g., Dataset Version 3, Version 4)

To address these requirements, a custom **Streamlit-based web application** was developed. The application integrates the YouTube Data API, the fine-tuned DistilBERT model checkpoint, manual relabeling functionality, and dataset export capabilities. This design transforms the sentiment classifier from a static research artifact into a practical tool for annotation, validation, and continuous dataset refinement.

System Architecture

The application follows a modular and linear processing pipeline:

1. The user provides a YouTube video URL
2. Comments are retrieved using the YouTube Data API
3. Comments are processed in batches using the fine-tuned DistilBERT model
4. Predicted sentiment labels and confidence scores are displayed in a Kanban-style interface
5. The user may manually reassign sentiment labels
6. The corrected data can be exported as a CSV file

This architecture ensures simplicity, scalability, and ease of human interaction.

Technologies Used

Python

The application is fully implemented in Python and integrates several widely adopted libraries.

Streamlit

Streamlit is used to construct the interactive web interface. It enables:

- Column-based layouts
- Button-driven sentiment reassignment
- Sliders for filtering and threshold selection
- CSV export functionality
- Session state management for reproducibility

Hugging Face Transformers

The Transformers library is used to load and execute the fine-tuned DistilBERT model checkpoint and to perform batch inference.

```
model = DistilBertForSequenceClassification.from_pretrained(
    os.path.abspath(MODEL_PATH),
    local_files_only=True
)
model.to(device)
model.eval()
```

PyTorch

PyTorch is employed for tensor operations, device management (CPU/GPU), and efficient inference.

```
with torch.no_grad():
    logits = model(**enc).logits
    probs = torch.softmax(logits, dim=1)
```

Google API Client

The Google API Client is used to retrieve YouTube comments and associated metadata, including:

- Comment text
- Like count
- Reply count
- Publication date

```
youtube = build("youtube", "v3", developerKey=API_KEY)
request = youtube.commentThreads().list(
    part="snippet",
    videoId=video_id,
    maxResults=100,
    textFormat="plainText"
)
```

Core Functionalities

Real-Time Sentiment Prediction

For computational efficiency, comments are processed in batches. For each comment, the model produces:

- A predicted sentiment label
- A confidence score
- The full probability distribution over the three sentiment classes

```
pred_ids = probs.argmax(axis=1)
confidence = probs.max(axis=1)
```

Kanban-Based Review Interface

Predicted comments are organized into three columns corresponding to sentiment polarity:

- Negative
- Neutral

- Positive

Each comment card displays the comment text (preview), number of likes, number of replies, publication date, and prediction confidence. Users can reassign sentiment labels through dedicated buttons.

```
if st.button("Neg", key=f"{cid}_neg"):
    st.session_state[f"move_{cid}"] = "negative"
```

A button-based reassignment mechanism was chosen instead of drag-and-drop due to stability limitations in Streamlit. This approach ensures faster interaction, reproducible behavior, and consistent session state management.

Filtering and Sorting

The application supports multiple filtering and sorting options, including:

- Minimum number of likes
- Minimum number of replies
- Maximum confidence threshold (to highlight uncertain predictions)
- Sorting by publication date, number of likes, number of replies, or lowest confidence

These features allow targeted review of influential comments, ambiguous predictions, and recent activity.

DATASET EXPORT

The application provides two dataset export formats:

Minimal Training Dataset

A lightweight CSV file containing only the information required for model retraining:

```
comment, sentiment
minimal_df = df[["comment", "label"]]
```

Extended Analytical Dataset

A comprehensive CSV file containing additional metadata, including predicted labels, corrected labels, confidence scores, full probability distributions, and engagement metrics. This format supports deeper error analysis and post-hoc evaluation.

Importance of the Web Application

The development of this application offers several key advantages:

1. **Real-World Model Validation**
   The model is evaluated on dynamic, real-world data rather than static test sets.
2. **Human-in-the-Loop Learning**
   Manual corrections enable iterative dataset improvement.
3. **Error Analysis**
   Misclassifications can be directly inspected and analyzed.
4. **Scalable Data Expansion**
   The application functions as a semi-automatic annotation tool.
5. **Deployment-Oriented Design**
   Demonstrates how a research model can be integrated into a practical end-user system.

Conceptual Significance

From a machine learning perspective, the application transforms a traditional supervised classifier into an interactive data refinement system. It closes the feedback loop between:

Model prediction → Human correction → Dataset improvement → Model retraining

This iterative process formed the foundation for the creation of Dataset Version 2 and the subsequent improvement in model performance.

The Streamlit-based web application serves simultaneously as:

- A real-time inference engine
- A manual annotation and correction tool
- A dataset generation pipeline
- A visualization and analysis dashboard
- A bridge between research experimentation and practical deployment

Overall, it demonstrates how a fine-tuned Transformer-based sentiment classifier can be embedded within an interactive system that supports continuous learning, dataset refinement, and real-world applicability.

Project Conclusion

This project explored the performance of a sentiment analysis model on a diverse set of user-generated comments, focusing on its accuracy, limitations, and potential areas for improvement. The analysis revealed that while the model performs reasonably well on clearly positive or negative statements, it struggles with nuanced, neutral, ironic, or context-dependent expressions. Key findings include:

1. Strengths of the Model:
    - High accuracy on comments with explicit positive or negative sentiment.
    - Effective detection of strong emotional language, especially when unambiguous keywords are present.
    - Consistent performance on comments with standard syntactic structure and formal language.
2. Limitations Identified:
    - Context sensitivity: The model often misclassifies comments when sentiment depends on context rather than individual words.
    - Neutral and subtle sentiment: Mildly positive, nostalgic, or neutral comments are frequently predicted as positive or negative.
    - Irony, humor, and emojis: These elements frequently mislead the model due to literal word interpretation.
    - Ambiguous or mixed messages: Comments containing both positive and negative elements present challenges, leading to lower confidence and misclassification.
3. Recommendations for Improvement:
    - Expand training data: Include more examples of ironic, humorous, neutral, and emoji-rich comments.
    - Context-aware modeling: Integrate methods that consider conversational context, comment threads, or surrounding text to improve nuanced understanding.
    - Semantic understanding of emotional words: Train the model to interpret emotionally charged terms in context, avoiding misclassification of supportive comments as negative.
    - Emoji-aware embeddings: Use specialized embeddings or sentiment scoring for emojis to capture informal digital communication more accurately.
    - Balanced dataset: Ensure adequate representation of neutral and subtly expressed sentiments to reduce bias toward strong positive/negative predictions.

Overall, the project demonstrates that while current sentiment analysis models are effective for clear-cut emotional content, they require additional context awareness, semantic understanding, and handling of informal language features to perform reliably in real-world social media environments. Implementing these improvements will enhance the model's robustness, reduce misleading predictions, and increase confidence in its sentiment classification for diverse user-generated content.

LITERATURE:

1. Siti Khomsah, “Sentiment Analysis On YouTube Comments Using Word2Vec and Random Forest,” *Jurnal Telematika*, vol. 18, no. 1, 2025.
2. Mohammed Sufyan et al., “Sentiment Analysis With YouTube Comments Using Deep Learning,” *Int. J. of Information Technology and Computer Engineering*, vol. 13, no. 2s, 2025.
3. Navishree Jain & Karuna Soni, “Extracting Sentiments from YouTube Comments Using Deep Learning and Transformers,” *IJRASET*, 2025.
4. Y. S. Ercikan & W. Elmasry, “YouTube Video Comments Sentiment Analysis Using Custom NLP Model,” *ICHORA 2025 Conference Proceedings*.
5. Jana Sorupaa et al., “YouTube Comment Sentiment Classification System,” *Journal of AI and Capsule Networks*, vol. 6, no. 1, 2024.
6. ArXiv Authors, “YTCommentVerse: A Multi-Category Multi-Lingual YouTube Comment Corpus,” *arXiv preprint*, 2025.

REFERENCE LINKS FROM LITERATURE REVIEW:

[1]: https://journals.mmupress.com/index.php/ijoras/article/view/1311

[2]: https://ieeexplore.ieee.org/document/10895870

[3]: https://www.sciencedirect.com/science/article/pii/S1877050925016783

[4]: https://conservancy.umn.edu/server/api/core/bitstreams/fd29baa3-4c71-4af2-9841-0be32e6a40a9/content

[5] https://arxiv.org/html/2509.11057v1